EXT 2/3

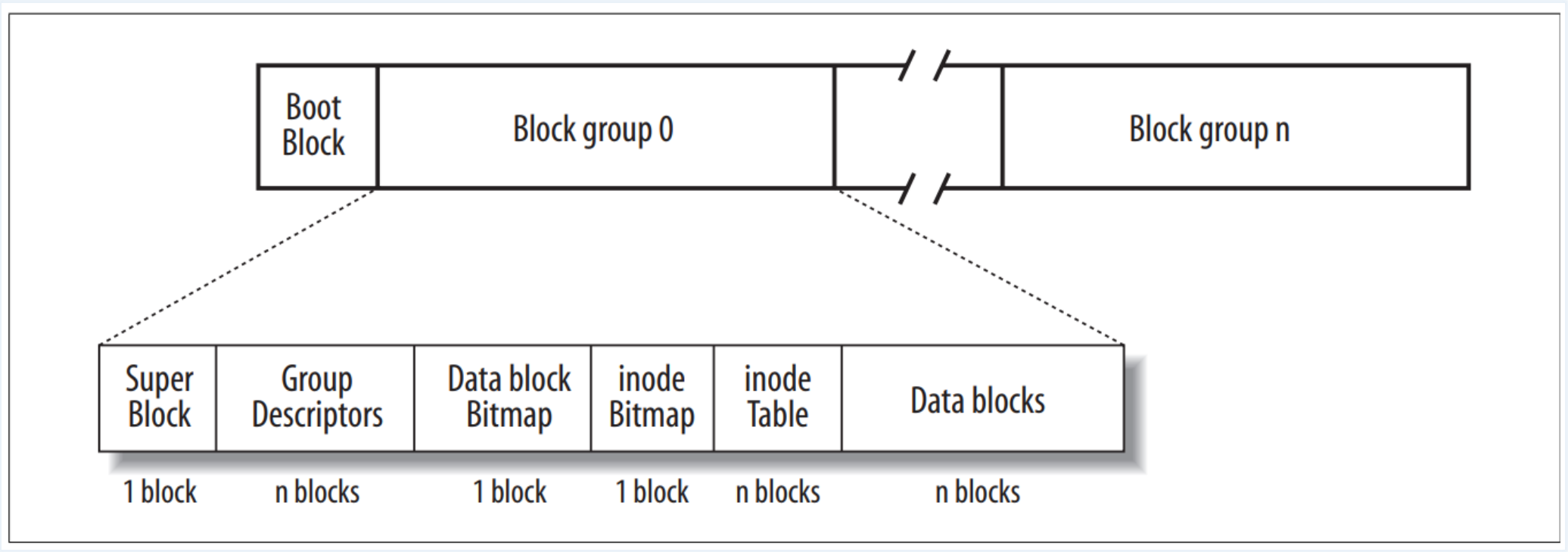
Overview

- Characteristics:
  - flexible: system administrator can choose:
    - block size (1K – 4K bytes), # of inodes per partition, etc.
    - sensible defaults
  - filesystem is partitioned into groups of blocks
    - each group stores both inodes and data blocks
    - rationale: keep these together to reduce seek time
  - preallocation of disk blocks to ensure contiguous files
  - short symbolic links (< 60 chars) are stored in the inode to speed pathname parsing
  - careful attention to crash semantics to support FS recovery
  - powerful consistency checker
    - `e2fsck`

EXT2/3 (2)

- Improvements considered, some support already exists:
  - block fragmentation to better accommodate small files
  - ACLs
  - transparent encryption and compression
  - undelete support
  - journaling [not implemented in ext2, instead new FS: ext3]
  - Now: ext4

EXT2: BASIC ON-DISK STRUCTURES



SUPERBLOCK

Type	Field	Description
__le32	s_inodes_count	Total number of inodes
__le32	s_blocks_count	Filesystem size in blocks
__le32	s_r_blocks_count	Number of reserved blocks
__le32	s_free_blocks_count	Free blocks counter
__le32	s_free_inodes_count	Free inodes counter
__le32	s_first_data_block	Number of first useful block (always 1)
__le32	s_log_block_size	Block size
__le32	s_log_frag_size	Fragment size
__le32	s_blocks_per_group	Number of blocks per group
__le32	s_frags_per_group	Number of fragments per group
__le32	s_inodes_per_group	Number of inodes per group
__le32	s_mtime	Time of last mount operation
__le32	s_wtime	Time of last write operation
__le16	s_mnt_count	Mount operations counter
__le16	s_max_mnt_count	Number of mount operations before check
__le16	s_magic	Magic signature
__le16	s_state	Status flag
__le16	s_errors	Behavior when detecting errors
__le16	s_minor_rev_level	Minor revision level
__le32	s_lastcheck	Time of last check
__le32	s_checkinterval	Time between checks
__le32	s_creator_os	OS where filesystem was created
__le32	s_rev_level	Revision level of the filesystem
__le16	s_def_resuid	Default UID for reserved blocks
__le16	s_def_resgid	Default user group ID for reserved blocks
__le32	s_first_ino	Number of first nonreserved inode
__le16	s_inode_size	Size of on-disk inode structure
__le16	s_block_group_nr	Block group number of this superblock
__le32	s_feature_compat	Compatible features bitmap
__le32	s_feature_incompat	Incompatible features bitmap
__le32	s_feature_ro_compat	Read-only compatible features bitmap
__u8 [16]	s_uuid	128-bit filesystem identifier
char [16]	s_volume_name	Volume name
char [64]	s_last_mounted	Pathname of last mount point
__le32	s_algorithm_usage_bitmap	Used for compression
__u8	s_prealloc_blocks	Number of blocks to preallocate
__u8	s_prealloc_dir_blocks	Number of blocks to preallocate for directories
__u16	s_padding1	Alignment to word
__u32 [204]	s_reserved	Nulls to pad out 1,024 bytes

BLOCK DESCRIPTOR & BITMAP

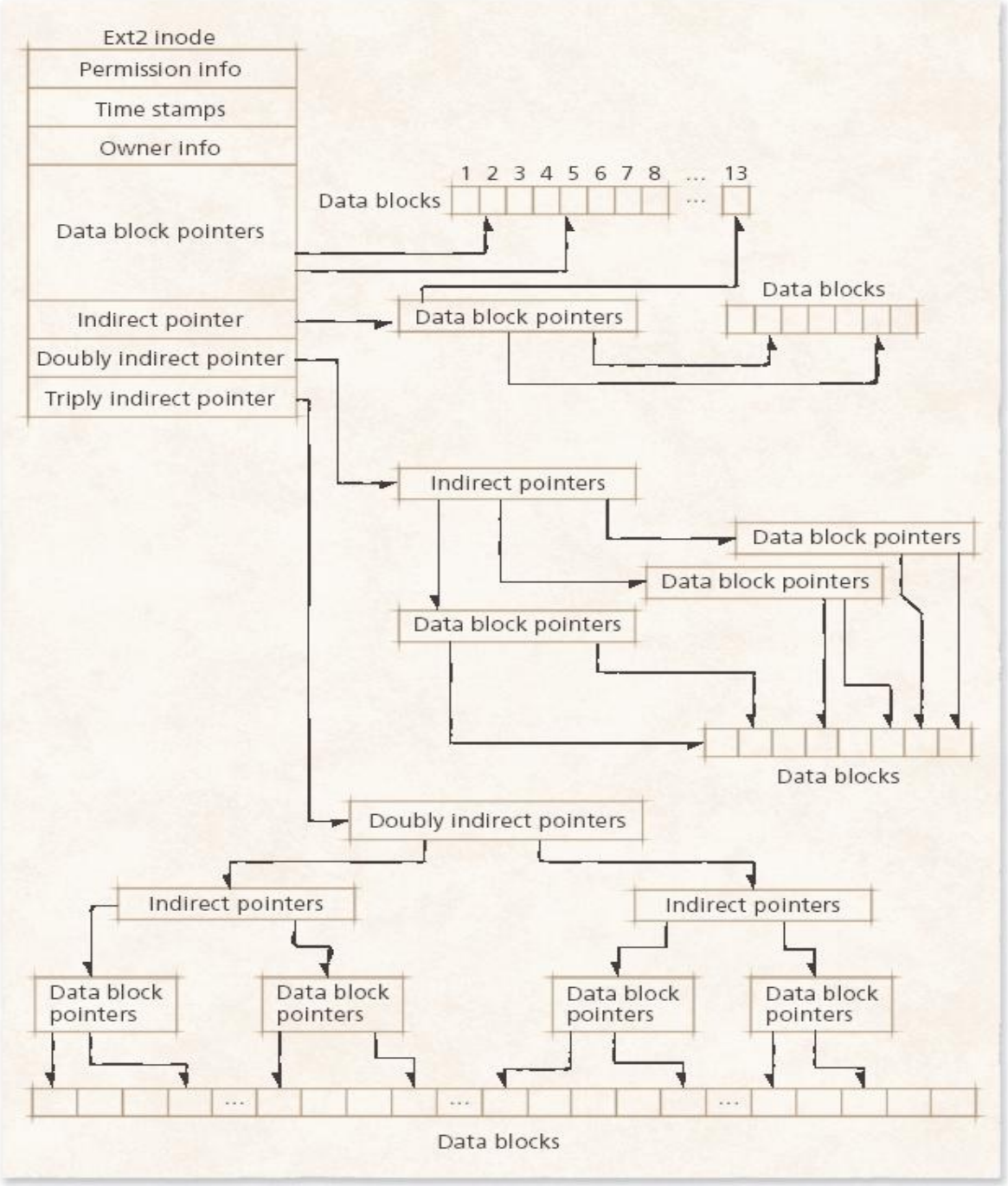
Type	Field	Description
__le32	bg_block_bitmap	Block number of block bitmap
__le32	bg_inode_bitmap	Block number of inode bitmap
__le32	bg_inode_table	Block number of first inode table block
__le16	bg_free_blocks_count	Number of free blocks in the group
__le16	bg_free_inodes_count	Number of free inodes in the group
__le16	bg_used_dirs_count	Number of directories in the group
__le16	bg_pad	Alignment to word
__le32 [3]	bg_reserved	Nulls to pad out 24 bytes

INODE TABLE

- Consecutive blocks; each has
  - » fixed number of inodes
- Inode
  - » 128 bytes
  - ➔ 8 per 1KiB block
  - ➔ **32 per 4KiB block**

Type	Field	Description
__le16	i_mode	File type and access rights
__le16	i_uid	Owner identifier
__le32	i_size	File length in bytes
__le32	i_atime	Time of last file access
__le32	i_ctime	Time that inode last changed
__le32	i_mtime	Time that file contents last changed
__le32	i_dtime	Time of file deletion
__le16	i_gid	User group identifier
__le16	i_links_count	Hard links counter
__le32	i_blocks	Number of data blocks of the file
__le32	i_flags	File flags
union	osd1	Specific operating system information
__le32 [EXT2_N_BLOCKS]	i_block	Pointers to data blocks
__le32	i_generation	File version (used when the file is accessed by a network filesystem)
__le32	i_file_acl	File access control list
__le32	i_dir_acl	Directory access control list
__le32	i_faddr	Fragment address
union	osd2	Specific operating system information

EXT2: INODE

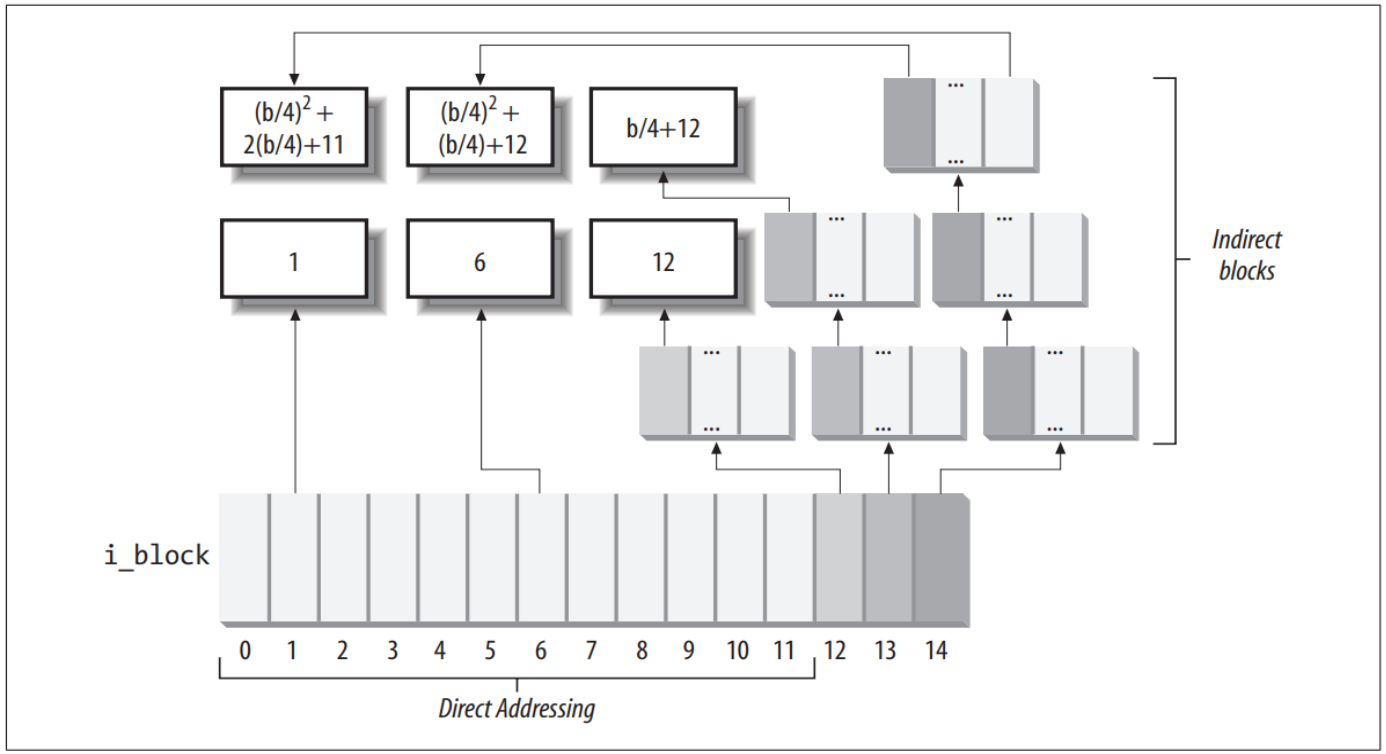

Ext2 inode
Permission info
Time stamps
Owner info
Data block pointers
Indirect pointer
Doubly indirect pointer
Triply indirect pointer
Data blocks
1 2 3 4 5 6 7 8 ... 13
Data block pointers
Data blocks
Indirect pointers
Data block pointers
Data block pointers
Data block pointers
Data blocks
Doubly indirect pointers
Indirect pointers
Indirect pointers
Data block pointers
Data block pointers
Data block pointers
Data block pointers
Data blocks

Ext2 Inode

```
├── Permission info
├── Time stamps
├── Owner info
│
├── Data block pointers [12]
│   ├── Data block 1
│   ├── Data block 2
│   ├── ...
│   └── Data block 12
│
├── Indirect pointer
│   └── Data block pointers [1024]
│       ├── Data block 13
│       ├── Data block 14
│       ├── ...
│       └── Data block 1024+12
│
└── Doubly indirect pointer
    └── Indirect pointers [1024]
        ├── Data block pointers [1024]
        │   ├── Data block
        │   ├── ...
        │   └── Data block
        ├── ...
        └── Data block pointers [1024]
            ├── Data block
            ├── ...
            └── Data block
```

```
├── Triply indirect pointer
│   └── Doubly indirect pointers [1024]
│       ├── Indirect pointers [1024]
│       │   ├── Data block pointers [1024]
│       │   │   ├── Data block
│       │   │   ├── ...
│       │   │   └── Data block
│       │   ├── ...
│       │   └── Data block pointers [1024]
│       │       ├── Data block
│       │       ├── ...
│       │       └── Data block
│       ├── ...
│       └── Indirect pointers [1024]
│           ├── Data block pointers
│           │   ├── Data block
│           │   ├── ...
│           │   └── Data block
│           ├── ...
│           └── Data block pointers [1024]
│               ├── Data block
│               ├── ...
│               └── Data block
├── . . .
```

BLOCK SIZE vs. MAX FILE SIZE

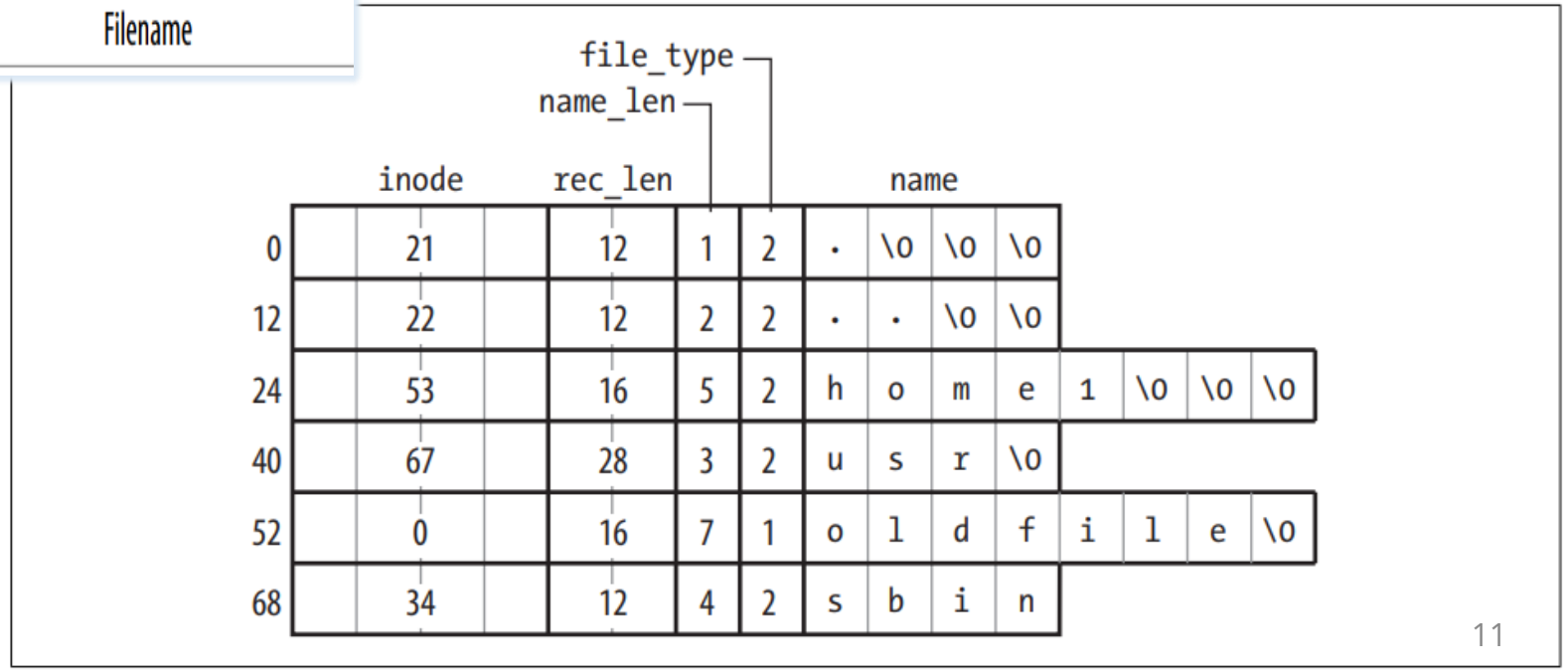


$$(b/4)^3+(b/4)^2+(b/4)+11$$

Block size	Direct	1-Indirect	2-Indirect	3-Indirect
1,024	12 KB	268 KB	64.26 MB	16.06 GB
2,048	24 KB	1.02 MB	513.02 MB	256.5 GB
4,096	48 KB	4.04 MB	4 GB	~ 4 TB

EXT2: DIRECTORY STRUCTURE

Type	Field	Description
__le32	inode	Inode number
__le16	rec_len	Directory entry length
__u8	name_len	Filename length
__u8	file_type	File type
char [EXT2_NAME_LEN]	name	Filename



EXT2: FILE TYPES

- Regular files
- Directories
- Symbolic links
  - » < 60 chars, store in inode, otherwise consumes an entire block
- Others: device file, pipe, socket
  - » No data blocks
  - » All info in inode

EXT3 CHARACTERISTICS

- Binary compatible with ext2 on-disk
- Reason for existence: huge disks == huge amounts of time to restore filesystem consistency after improper shutdown
- (Must check inodes, etc.)
- Major improvement over ext2: log that stores info about in-progress file operations
- On boot, can check log and quickly restore filesystem consistency
- Journaling filesystems: DANGER!

Creation of ext2 Filesystem

- Hard drives already have low-level format
- **`mke2fs`** does the work
  - Can choose blocksize, # of inodes, reserved blocks [superuser] %
  - Initialize superblock
  - Initialize group descriptors
  - Build block groups
  - Set to zero bitmaps for inodes, data blocks
  - Check for bad blocks [can be skipped]
  - Create **/**
  - Create '**lost+found**', where bad blocks go

Floppy ext2 Creation

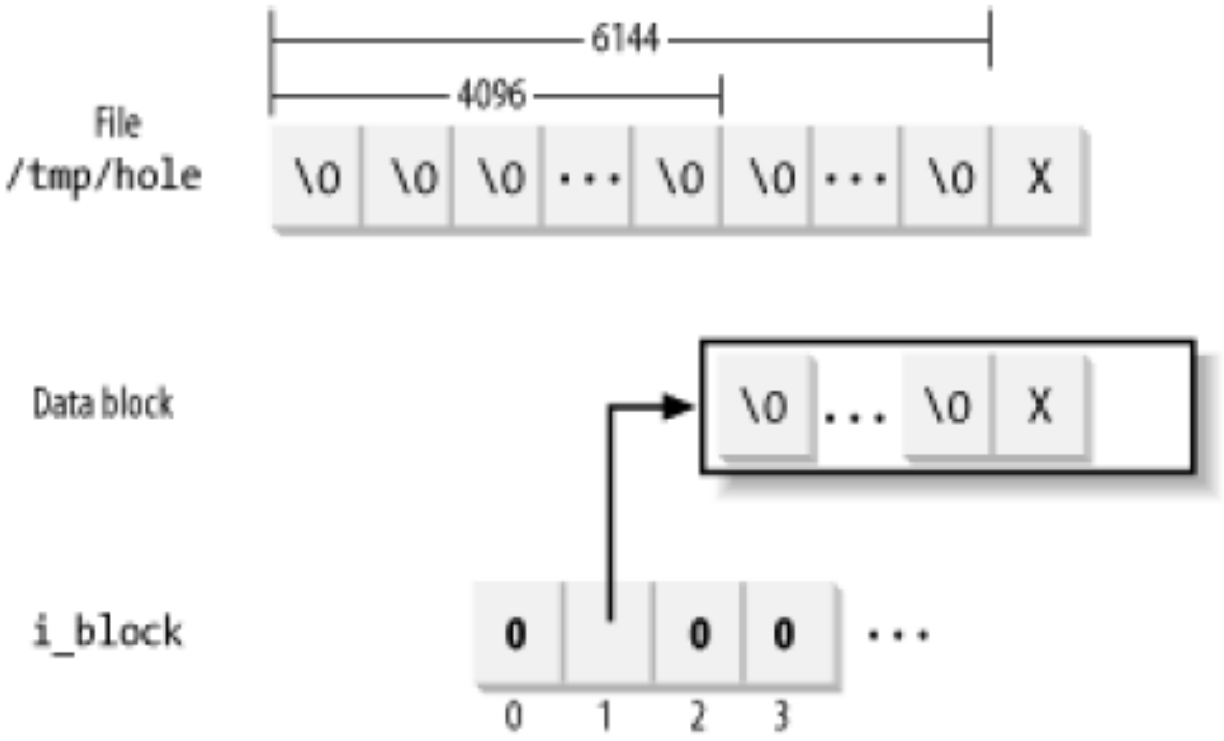
Table 17-7. Ext2 block allocation for a floppy disk

Block	Content
0	Boot block
1	Superblock
2	Block containing a single block group descriptor
3	Data block bitmap
4	Inode bitmap
5-49	Inode table: inodes up to 10: reserved; inode 11: *lost+found*; inodes 12-360: free
50	Root directory (includes `.`, `..`, and *lost+found*)
51	*lost+found* directory (includes `.` and `..`)
52-62	Reserved blocks preallocated for *lost+found* directory
63-1439	Free blocks

From: Understanding the Linux Kernel (Bovet/Cesati)

File Holes in ext2/3

Figure 17-6. A file with an initial hole



Straightforward implementation: zeros in inode block list correspond to unallocated blocks

From: Understanding the Linux Kernel (Bovet/Cesati)

JOURNALING BASICS

- Idea (*a.k.a.*, **write-ahead log**, WAL):
  - copy blocks to be written FIRST to the journal, then to the filesystem
  - when blocks have successfully been written to filesystem delete from journal
  - group low-level operations together into high-level operations (transactions)
- Two possibilities on filesystem recovery:
  - system failed before high level operation was committed to journal
    - do **not** commit these changes to the filesystem
  - system failed after high level operation was committed to journal
    - **do** commit these changes to the filesystem

Three levels of journaling

- **Journal** mode:
  - write **ALL DATA** and metadata blocks to journal before writing to filesystem
  - most expensive
  - highest consistency guarantees
- **Ordered** mode:
  - write **ONLY METADATA** blocks to journal; write **data** blocks to disk **before metadata** blocks
  - minimizes file damage
  - default ext3 mode; usually the best performance/consistency trade off,
- **Writeback** mode
  - write only metadata blocks to journal
  - least performance penalty
  - lowest consistency guarantee

EXT4

- Large file system support
  - volumes up to 1EB
  - files up to 1TB
- Employs **extents**
  - ranges of contiguously allocated blocks
  - improves performance for large files
  - four extents per *inode*, each can address 128MB
- Compatibility
  - compatibility with ext2/3
    - unless extents are used

EXT4 (2)

- Persistent pre-allocation for disk files
- Delayed space allocation
- Large # of subdirectories supported
  - 64,000 instead of 32,000 in ext3
- Journal checksumming
- Faster file system checking
  - unallocated block groups and inode table sections are marked and skipped
- Improved timestamps
  - **nanosecond** granularity